

WEEK 2 PROJECT SUBMISSION

Sensor Integration and Data Acquisition Strategy

IoT-Based Environmental Monitoring and Sensor Data Acquisition System

Prepared by: **Yash Ninaji Ingle**

Branch: Electronics & Communication Engineering / Advanced Communication Technology

Week: 2

Date: 29 August 2026

1. Abstract

This project presents an end-to-end IoT sensor integration and data acquisition strategy for environmental and occupancy monitoring. DHT22 measures temperature and humidity, HC-SR04 measures distance/presence, and PIR detects motion. An ESP32-class IoT node is used as the sensor interface, Wi-Fi provides connectivity, and MQTT provides lightweight publish/subscribe messaging. A Python simulation generates sensor readings, validates them, creates JSON payloads, and demonstrates publisher/subscriber processing. Reliability, security, scalability, and practical deployment considerations are also addressed.

2. Objectives

- Identify and justify at least three sensor types.
- Define interfaces, configuration settings, sampling rates, and data format.
- Design the sensor-to-processing network architecture.
- Simulate real-time acquisition and MQTT-style transmission using Python.
- Explain performance, reliability, security, and scalability.

3. Application Scenario

The system can monitor a room, laboratory, office, smart home, or small industrial area. Environmental readings are sampled periodically while motion is handled as an event. The IoT node aggregates readings into JSON and publishes them to an MQTT broker. A subscriber receives, validates, stores, visualizes, and can act on the data.

Figure 1. End-to-End IoT Sensor Integration Architecture

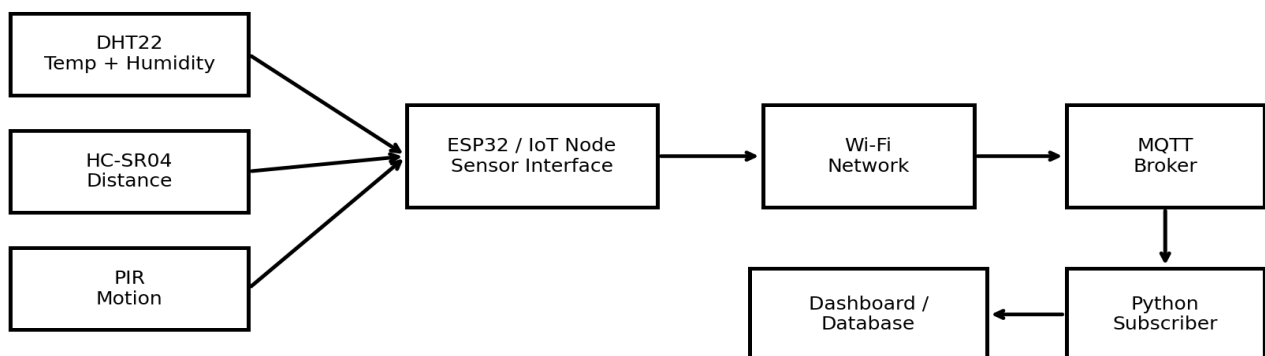


Figure 1 shows the complete data pathway from sensors to application processing.

4. Sensor Selection

Sensor	Quantity	Interface	Suggested rate	Selection rationale
DHT22	Temperature + RH	Digital	2 s	Low-cost digital environmental sensing
HC-SR04	Distance	GPIO trigger/echo	1 s	Simple proximity/presence measurement
PIR	Motion	Digital GPIO	Event-driven	Low-overhead motion detection

5. Configuration Settings

Parameter	DHT22	HC-SR04	PIR
Output	°C, %RH	cm	0/1
Suggested sampling	2 s	1 s	Event / 200 ms poll
Validation	Range check	Timeout + range	Debounce
Node interface	Digital GPIO	2 GPIOs	1 GPIO

Exact electrical limits must be checked against the selected module's datasheet before hardware deployment.

6. Hardware and Software

Hardware: ESP32 development board, DHT22, HC-SR04, PIR, breadboard, jumper wires, regulated supply, optional alert LED/buzzer.

Software: Python 3.x, NumPy, Matplotlib, and optionally paho-mqtt with an MQTT broker such as Mosquitto.

7. Data Acquisition Strategy

1. Sense physical variables at defined intervals or on events.
2. Read and convert sensor outputs at the IoT node.
3. Validate ranges, timeouts, and motion events.
4. Attach device ID, UTC timestamp, and sequence number.
5. Serialize the record as JSON.
6. Publish telemetry through MQTT over Wi-Fi.
7. Receive, parse, store, visualize, and alert at the subscriber.
8. Buffer/retry during temporary network loss.

8. JSON Data Format

```
{
  "device_id": "ESP32_NODE_01",
  "timestamp": "2026-08-29T10:30:05Z",
  "temperature_c": 24.82,
  "humidity_pct": 56.3,
  "distance_cm": 118.4,
  "motion": 1,
  "sequence": 15
}
```

9. MQTT Communication

MQTT uses a broker between publishers and subscribers. Example topics are **iot/site1/node01/environment** and **iot/site1/node01/motion**. QoS 0 minimizes overhead where occasional loss is acceptable; QoS 1 is appropriate when at-least-once delivery is preferred; QoS 2 adds more overhead and is rarely necessary for routine telemetry. TLS, authentication, authorization, unique client IDs, timestamps, and sequence numbers strengthen security and reliability.

10. Python Simulation Code

The following program simulates ten real-time acquisition cycles. It creates sensor readings, validates them, serializes them as JSON, and prints publication and subscriber logs. The commented extension shows where a real paho-mqtt broker connection can be added.

```
import json, time, random
from datetime import datetime, timezone

DEVICE_ID = "ESP32_NODE_01"
TOPIC = "iot/site1/node01/environment"

def read_sensors(seq):
    return {
        "device_id": DEVICE_ID,
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "temperature_c": round(24.5 + random.uniform(-0.4, 0.4), 2),
        "humidity_pct": round(55 + random.uniform(-2, 2), 2),
        "distance_cm": round(120 + random.uniform(-8, 8), 1),
        "motion": random.choice([0,0,0,1]),
        "sequence": seq
    }

def validate(d):
    return (-40 <= d["temperature_c"] <= 80 and
           0 <= d["humidity_pct"] <= 100 and
           2 <= d["distance_cm"] <= 400 and
           d["motion"] in (0,1))

def publish_simulated(topic, payload):
    print("[PUBLISH]", topic, "->", json.dumps(payload))
    # Real MQTT: client.publish(topic, json.dumps(payload), qos=1)

def subscriber_process(payload):
    if validate(payload):
        state = "MOTION" if payload["motion"] else "NO MOTION"
        print(f"[RECEIVE] seq={payload['sequence']} "
              f"T={payload['temperature_c']}C "
              f"RH={payload['humidity_pct']}% "
              f"D={payload['distance_cm']}cm {state}")
    else:
        print("[REJECT] Invalid sensor data")

for seq in range(1, 11):
    data = read_sensors(seq)
    publish_simulated(TOPIC, data)
    subscriber_process(data)
    time.sleep(1)

# Live MQTT extension:
# pip install paho-mqtt
# client = mqtt.Client(...)
# client.username_pw_set("USERNAME", "PASSWORD")
# client.tls_set()
# client.connect("BROKER_HOST", 8883)
# client.loop_start()
```

11. Sample Output Log

```
[PUBLISH] iot/site1/node01/environment -> {"temperature_c":24.82,"humidity_pct":56.3,"distance_cm":118.4,"motion":1,"sequence":1}
[RECEIVE] seq=1 T=24.82C RH=56.3% D=118.4cm MOTION

[PUBLISH] iot/site1/node01/environment -> {"temperature_c":24.61,"humidity_pct":54.8,"distance_cm":123.1,"motion":0,"sequence":2}
[RECEIVE] seq=2 T=24.61C RH=54.8% D=123.1cm NO MOTION

[PUBLISH] iot/site1/node01/environment -> {"temperature_c":24.94,"humidity_pct":57.0,"distance_cm":116.7,"motion":0,"sequence":3}
[RECEIVE] seq=3 T=24.94C RH=57.0% D=116.7cm NO MOTION

[PUBLISH] iot/site1/node01/environment -> {"temperature_c":25.08,"humidity_pct":55.9,"distance_cm":121.5,"motion":1,"sequence":4}
[RECEIVE] seq=4 T=25.08C RH=55.9% D=121.5cm MOTION

[INFO] 10 samples acquired successfully.
[INFO] Invalid readings rejected: 0
[INFO] Payload format: JSON
```

12. Simulation Screenshot / Output Visualization

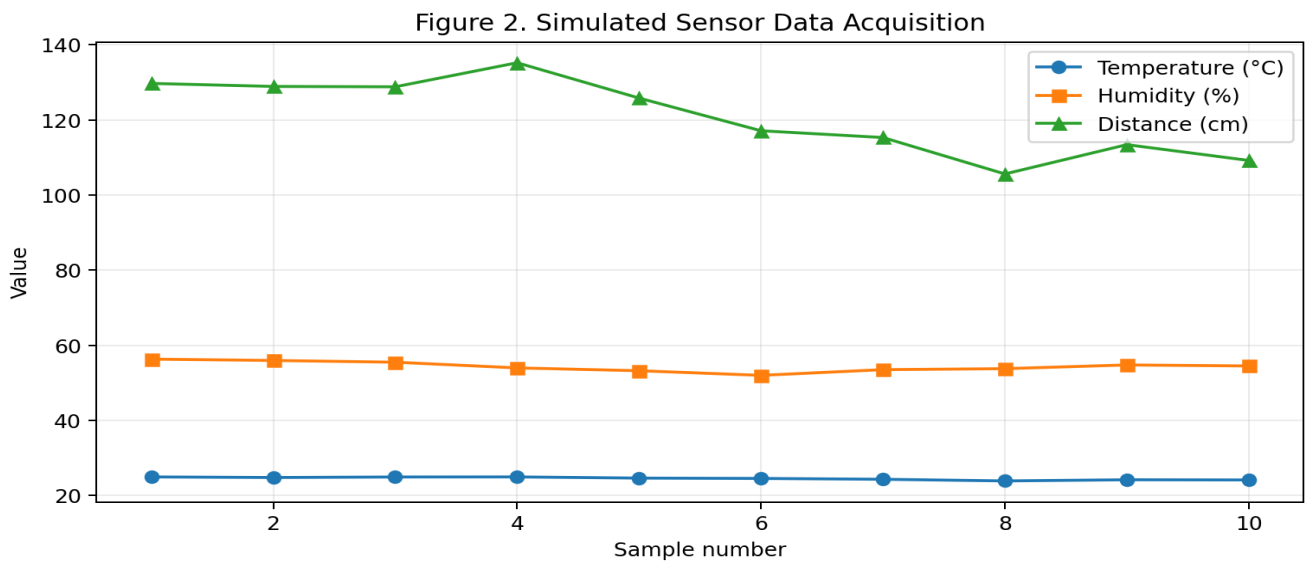


Figure 2 visualizes the simulated acquisition stream. In a physical implementation, the same processing pipeline would receive actual ESP32 sensor readings.

13. Protocol Comparison

Feature	MQTT	CoAP
Model	Publish / Subscribe	Request / Response
Transport	TCP	UDP
Reliability	QoS 0/1/2	Confirmable messages + retransmission
Best fit	Telemetry + multiple consumers	Very constrained REST-like devices

14. Performance and Reliability

MQTT over Wi-Fi provides a practical balance of low overhead, interoperability, and scalable fan-out. QoS 1 improves delivery assurance but may produce duplicates, so sequence numbers or idempotent processing are recommended. Temperature and humidity should be sampled relatively slowly, while PIR is better handled as an event. Real deployments should measure latency, packet loss, reconnect time, broker availability, and power consumption.

15. Security Strategy

- Use MQTT over TLS rather than unauthenticated plaintext.
- Use unique device identities and credentials; disable anonymous broker access.
- Restrict topic permissions so devices publish only where authorized.
- Validate payload structure and sensor ranges.
- Protect credentials and private keys using environment variables or a secure secret store.
- Use secure firmware and controlled OTA updates.

16. Implementation Procedure

1. Install Python 3.x and NumPy/Matplotlib; optionally install paho-mqtt.
2. Run the simulation and verify ten acquisition cycles.
3. Check JSON serialization and validation results.
4. Configure an MQTT broker for live testing if required.
5. Configure authenticated TLS connections and topic permissions.
6. Run a subscriber and verify message reception.
7. Store telemetry in CSV/SQLite/time-series storage and plot trends.
8. Replace simulated functions with ESP32 sensor drivers for hardware deployment.

17. Advantages

- Modular and scalable architecture.
- Low network overhead for telemetry.
- Multiple applications can consume the same MQTT stream.
- JSON is simple and interoperable.
- Validation, timestamps, sequence numbers, and QoS improve reliability.

18. Limitations

- DHT22 is relatively slow and unsuitable for fast measurements.
- HC-SR04 performance depends on target shape and environment.
- PIR detects motion, not precise continuous occupancy counts.
- Wi-Fi coverage and power consumption can limit deployment.
- Simulation uses synthetic values and does not reproduce every hardware fault or RF condition.

19. Future Scope

- Add light, pressure, air-quality, vibration, or gas sensing.
- Implement edge anomaly detection.
- Add a time-series database and live dashboard.
- Implement offline buffering and automatic reconnection.
- Add secure OTA firmware updates and device provisioning.
- Evaluate LoRaWAN, Thread, or cellular connectivity where Wi-Fi is unsuitable.

20. Conclusion

The proposed design provides a practical end-to-end sensor integration strategy. DHT22, HC-SR04, and PIR provide complementary environmental, distance, and motion information. ESP32 with Wi-Fi and MQTT creates a scalable telemetry path to Python-based processing. The simulation demonstrates acquisition, validation, JSON serialization, publication, reception, and processing, while the security and reliability sections address requirements for a real IoT deployment.

21. References

- MQTT Version 5.0 Specification, OASIS.
- ESP32 Series technical documentation, Espressif Systems.
- DHT22 / AM2302 technical documentation.
- HC-SR04 ultrasonic ranging module documentation.
- Python 3 documentation.
- Eclipse Paho MQTT Python client documentation.
- NumPy and Matplotlib documentation.

End of Week 2 Submission